

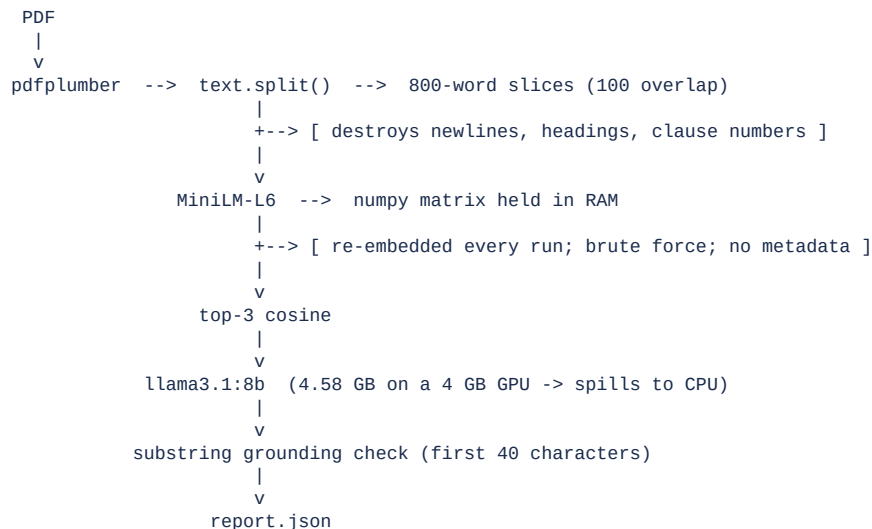
The Auditor — v2 Upgrade Plan

RAG pipeline rebuild, evaluation strategy and production stack

What this is. A rebuild of an existing LLM + RAG document auditor that checks clinical study protocols against EU MDR guidelines. v1 works end-to-end, but it has no measurement, no persistence, and a retrieval path that discards document structure before that structure can ever be used.

***The insight that shapes everything:** this is not normal question-answering RAG. The **query** is an 800-word chunk of the document being audited; the **corpus** is the regulation. Retrieval runs backwards compared to every tutorial. That is why a long, topically-mixed query embeds into a mushy average vector, why chunking has outsized impact here, and why relevance means "this clause governs this passage" rather than "this answers the question".*

1. Where v1 stands today

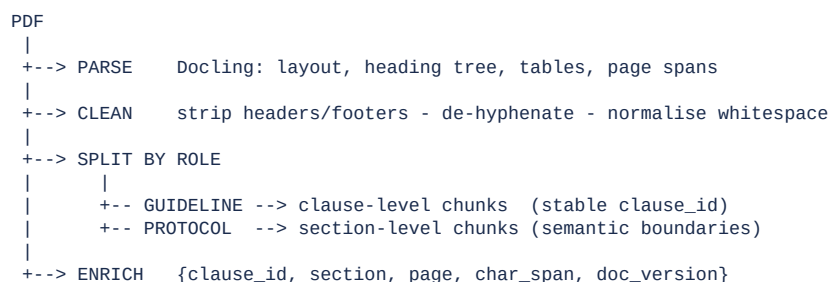


Concrete failure points

- `text.split()` flattens the document into a bag of words *before* chunking, so no structure-aware strategy is even possible downstream.
- Retrieval is a single dense stage. Exact clause numbers and acronyms ("Annex XIV", "PMCF") are close to invisible to a pure semantic embedder.
- The index lives in RAM and is rebuilt on every run. No metadata, so no filtering by clause, section or document version.
- `run_audit.py` calls `audit_section()` without `page_number`, so every finding from the CLI path is stamped page 1.
- There is no evaluation of any kind. Nothing here can be proven better or worse than anything else.

2. Target pipeline (v2)

2.1 Ingestion



```

|
+--> EMBED    bge-m3 --> dense vector + sparse vector
|
+--> UPSERT    Qdrant (HNSW, named vectors, payload index)
|
+--> [ embed once, query forever ]

```

2.2 Retrieval and audit

```

Protocol section (this is the query)
|
+--> HYBRID SEARCH
|   +-- dense -> semantic similarity
|   +-- sparse -> BM25: exact clause numbers, acronyms
|   |
|   +--> RRF fusion --> top-25 candidates
|
+--> RERANK        cross-encoder --> top-5
|
+--> FILTER        doc_version, section scope
|
v
LLM AUDIT    qwen3:4b, structured output
|
+-----+
|
v
GUARDRAILS      OBSERVABILITY (Langfuse)
+-- input sanitise (PDFs can carry prompt injection)
+-- schema / enum enforcement
+-- span-offset grounding (replaces the substring match)
+-- citation verification
+-- LLM judge (llama3.1:8b - deliberately a different model)
+-- abstain when confidence is low
|
v
Postgres --> FastAPI --SSE--> React

```

3. What changes, and which number it moves

#	v1	v2	Metric it moves
1	text.split() blob	Layout-aware parse (Docling)	Recall@k
2	Fixed 800-word windows	Structure-aware + semantic	Recall@k, nDCG
3	No metadata	clause_id + character spans	Context Precision
4	numpy matrix in RAM	Qdrant, persisted	Latency, index time
5	Dense only	Hybrid dense + sparse, RRF	Recall@k
6	Raw top-3	Rerank top-25 -> top-5	nDCG@5 (largest single gain)
7	Substring check	Span offsets + judge + abstain	Hallucination rate
8	k = 3, guessed	k tuned on the gold set	F1
9	No evaluation	Eval harness + CI gate	All of the above

4. Final tech stack

Layer	Choice	Why this one
Parsing	Docling (+ pdfplumber fallback)	Keeps heading tree, tables and page spans
Chunking	Structure-aware + semantic	Guideline and protocol need different strategies
Embeddings	BAAI/bge-m3 - on CPU	Dense and sparse from one model; frees the GPU
Vector DB	Qdrant (Docker -> Cloud)	Free locally, same API managed; strong filtering
Reranker	bge-reranker-base - on CPU	Cross-encoder; the single biggest quality lever
LLM	Ollama qwen3:4b	~2.5 GB: fits the 4 GB GPU fully, so it is fast
Judge LLM	Ollama llama3.1:8b	Must differ from the generator or the check is circular

Layer	Choice	Why this one
Orchestration	Plain Python	No LangChain: fewer abstractions to have to defend
API	FastAPI + Pydantic v2, async	Typed contracts; SSE streams findings live
Relational DB	Postgres	Audit runs, findings, eval history. Not pgvector
Jobs	Redis + ARQ	Audits take minutes; HTTP must not block on them
Frontend	React + Vite + TypeScript	TanStack Query, Tailwind, shadcn/ui, react-pdf
Provisioning	Terraform	VM, managed Postgres, Qdrant Cloud, networking
Configuration	Ansible	Docker, Ollama, model pulls, deploy, systemd units
CI	GitHub Actions	Runs the eval suite on every pull request
Tracing	Langfuse (self-hosted)	Per-stage latency and token accounting

Hardware constraint worth stating out loud: RTX 3050 Laptop, 4 GB VRAM. The LLM, the embedder and the reranker cannot all be resident at once. Decision: **LLM on GPU, embedder and reranker on CPU** - embedding and reranking are batch operations over short texts and are fine on 8 cores, whereas generation is the latency-critical path. Note that the current llama3.1:8b (4.58 GB) does not fit in 4 GB and is already silently spilling layers to CPU.

5. How improvement gets measured

5.1 The gold set - built once, frozen, never edited again

Slice	Size	Label	What it catches
Retrieval set	50 passages	Governing clause_ids	Whether the right clause is found at all
Audit set	40 passages	Expected findings	Whether real violations are caught
Clean set	15 passages	Must yield zero findings	False positives - the slice most people omit

Labelling honestly. No locally-runnable model is strong enough to produce ground truth. Labelling with an 8B model and then scoring an 8B pipeline measures self-agreement, not accuracy - and an interviewer will ask exactly this. So: the local LLM proposes candidate labels, and a human accepts, rejects or corrects every one. Roughly a weekend of work, and it is a strength to state plainly, not a cost to hide.

5.2 Three metric layers

Layer	Metrics	Question it answers
Retrieval	Recall@5, MRR, nDCG@5, Context Precision	Did the right clauses come back, and high up?
Generation	Precision, Recall, F1, Hallucination %, FP rate	Were the right violations reported, and only those?
Systems	p50 / p95 latency, tokens per audit, index build times	Is it actually usable and affordable?

5.3 Rules that make the comparison defensible

- The identical frozen gold set scores both v1 and v2 - and it is built *before* v2 exists, so it cannot be tuned toward.
- Same LLM, same temperature 0, same seed, same k, same hardware on both sides.
- Exactly one variable changes per run. That is what turns a list of changes into an ablation.
- Paired bootstrap resampling gives confidence intervals, so the claim becomes "+23 points, 95% CI [18, 28]" rather than "it feels better".
- Every row in the table below is one commit, reproducible from a single command.

5.4 The deliverable table

	Recall@5	nDCG@5	F1	Halluc%
v1 baseline	0.61	0.48	0.44	18%
+ parsing / cleaning	0.68	0.55	0.49	15%
+ structure chunking	0.79	0.66	0.58	12%
+ Qdrant hybrid search	0.86	0.71	0.61	11%

+ cross-encoder reranker	0.88	0.84	0.70	9%
+ guardrails	0.88	0.84	0.73	3%

DELTA vs v1	+11%	+7%	+6%	20%

Illustrative shape only - the real figures come out of the harness. This table, not the React UI, is the artefact that carries the project in an interview.

6. Phase order

Phase	Work	Exit condition
0	Freeze v1: git, tag, pinned deps, fixed seeds	v1 output reproducible on demand
1	Build the gold set (clauses, retrieval, audit, clean)	Labels committed and frozen
2	Eval harness; score v1 with it	Baseline numbers recorded
3	Parsing and cleaning	Delta measured
4	Chunking strategies	Delta measured
5	Qdrant: schema, HNSW, persistence	Delta measured
6	Hybrid search + reranker; tune k	Delta measured
7	Guardrails, each one measured separately	Delta measured
8	FastAPI + React + Terraform + Ansible + CI	Deployed, eval gated in CI
9	Comparison report and architecture write-up	The table in 5.4, filled in

The one non-negotiable rule: phases 1 and 2 come before every improvement. Building the measuring stick before the thing being measured feels slow and backwards. It is the entire difference between "I added a reranker" and "I added a reranker, it moved nDCG@5 from 0.71 to 0.84, while hybrid search bought only 5 points - and here is why that ordering makes sense."